

Perplexity.

Cross-entropy loss is measured in nats, which is hard to interpret directly. Exponentiating it gives *perplexity*: the effective number of equally-likely tokens the model is choosing among at each step. This topic derives perplexity as a geometric-mean inverse probability, relates it to bits-per-token, and computes it in full — nothing summarised or deferred.

THE EQUATION

$$\text{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p(x_t | x_{<t})\right) = \left(\prod_{t=1}^T p(x_t | x_{<t})\right)^{-1/T}$$

Worked example. The four per-token probabilities from Topic 1, (0.6104, 0.5587, 0.4753, 0.4644): cross-entropy in nats and bits, and $\text{PPL} = 1.9091$ — the model is effectively choosing between about two tokens per step.

Topic 2 of 3 in this module · companion to the course page · v1.0

Contents

1	From loss to perplexity	2
2	Bits per token	3
3	The uniform baseline: PPL = V	3
4	Worked computation (real numbers)	3
5	Properties / consequences that matter	4
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 From loss to perplexity

Topic 1 left us with the per-token negative log-likelihood, $\mathcal{L} = -\frac{1}{T} \sum_t \log p(x_t | x_{<t})$, measured in nats. Perplexity is simply its exponential, which puts the number back on the scale of *probabilities* (a count of tokens) rather than log-probabilities.

$$\text{PPL} = \exp(\mathcal{L}) = \exp\left(-\frac{1}{T} \sum_{t=1}^T \log p(x_t | x_{<t})\right) \quad (1)$$

The exponential of a negative average log undoes the logarithm and the averaging. Pulling them out one at a time:

$$\exp\left(-\frac{1}{T} \sum_t \log p_t\right) = \exp\left(\sum_t \log p_t^{-1/T}\right) = \prod_t p_t^{-1/T} = \left(\prod_{t=1}^T p_t\right)^{-1/T},$$

where we abbreviate $p_t \equiv p(x_t | x_{<t})$. So perplexity is the *inverse geometric mean* of the per-token probabilities:

$$\text{PPL} = \left(\prod_{t=1}^T p(x_t | x_{<t})\right)^{-1/T} = \frac{1}{\sqrt[T]{p_1 p_2 \cdots p_T}} \quad (2)$$

Suppose the model were perfectly indifferent among k tokens and assigned probability $1/k$ to each. Then every $p_t = 1/k$ and (2) gives $\text{PPL} = k$. So perplexity is the *effective branching factor*: the size of the uniform vocabulary the model is, on average, just as uncertain as. $\text{PPL} = 2$ means “about as unsure as a fair coin flip per token”; $\text{PPL} = 1$ means perfect certainty.

2 Bits per token

Switching the logarithm to base 2 measures the same uncertainty in *bits* instead of nats. Define

$$\mathcal{L}_{\text{bits}} = -\frac{1}{T} \sum_{t=1}^T \log_2 p(x_t | x_{<t}) = \frac{\mathcal{L}}{\ln 2},$$

since $\log_2 x = \ln x / \ln 2$. Then perplexity can equally be written as a power of two:

$$\boxed{\text{PPL} = 2^{\mathcal{L}_{\text{bits}}} = e^{\mathcal{L}}} \quad (3)$$

Bits-per-token has a coding interpretation: it is the average number of bits an optimal compressor using the model’s distribution would spend per token. A model with $\mathcal{L}_{\text{bits}} = 1$ needs one bit per token on average ($\text{PPL} = 2$); halving the bits squares nothing — it *square-roots* the perplexity, because $\text{PPL} = 2^{\mathcal{L}_{\text{bits}}}$ is exponential in the bit count.

3 The uniform baseline: $\text{PPL} = V$

A model that has learned nothing assigns the uniform distribution $p_t = 1/V$ to every token. Its loss is $\mathcal{L} = -\log(1/V) = \log V$ and so

$$\text{PPL} = \exp(\log V) = V.$$

This is the anchor for interpreting any perplexity number: the uniform model over GPT-2’s $V = 50,257$ tokens has $\text{PPL} = 50,257$. Real GPT-2 small scores roughly $\text{PPL} \approx 29$ on its held-out data — it has collapsed an effective 50,257-way choice down to a ~ 29 -way one.

4 Worked computation (real numbers)

Reuse the four per-token probabilities the model assigned to the true tokens in Topic 1 (**the cat sat on**):

$$(p_1, p_2, p_3, p_4) = (0.610405, 0.558696, 0.475332, 0.464426), \quad T = 4.$$

4.1 Step A — the product and its inverse geometric mean

$$\prod_t p_t = 0.610405 \times 0.558696 \times 0.475332 \times 0.464426 = 0.075285.$$

$$\text{PPL} = \left(\prod_t p_t \right)^{-1/4} = 0.075285^{-0.25} = \boxed{1.909076}.$$

4.2 Step B — cross-entropy in nats and bits

Per-token surprisals (in bits, $-\log_2 p_t$):

t	1	2	3	4	average
p_t	0.6104	0.5587	0.4753	0.4644	—
$-\log p_t$ (nats)	0.4936	0.5821	0.7437	0.7670	$\mathcal{L} = 0.6466$
$-\log_2 p_t$ (bits)	0.7122	0.8399	1.0730	1.1065	$\mathcal{L}_{\text{bits}} = 0.9329$

Check the conversions: $\mathcal{L}_{\text{bits}} = \mathcal{L} / \ln 2 = 0.6466 / 0.6931 = 0.9329$, and both routes to perplexity agree, $e^{0.6466} = 2^{0.9329} = 1.9091$. (It equals the inverse geometric mean from Step A, as it must.)

4.3 Step C — read it as an effective vocabulary

PPL = 1.91 on this sequence means the model is, on average, as uncertain as if it were picking uniformly among about 1.9 tokens per step — close to a coin flip, far better than the $V = 5$ uniform baseline (PPL = 5) for this toy vocabulary.

	Loss $\rightarrow$ perplexity (intensity scales with PPL, capped at 80)						
$\mathcal{L}$ (nats)	0.0	0.5	0.6466	1.0	1.5	2.0	4.0
PPL = $e^{\mathcal{L}}$	1.00	1.65	1.91	2.72	4.48	7.39	54.60

The relationship is exponential: each extra nat of loss multiplies perplexity by $e \approx 2.718$, so small loss differences mean large perplexity differences once the loss is high.

Perplexity is loss on a more legible scale. $\text{PPL} = \exp(\mathcal{L})$ converts “average nats of surprise” into “effective number of choices.” Lower is always better; PPL = 1 is perfect, and the uniform model sets the ceiling at PPL = V .

5 Properties / consequences that matter

- Monotone in the loss.** $\text{PPL} = \exp(\mathcal{L})$ is strictly increasing, so minimising cross-entropy and minimising perplexity are the same optimisation — perplexity is just the report card.
- Geometric, not arithmetic, mean.** One very low-probability token (a near-zero p_t) sends the product toward 0 and the perplexity toward ∞ . Perplexity is dominated by the model’s *worst* predictions, not its average-looking ones.
- Range.** $1 \leq \text{PPL} \leq V$ for any valid distribution over V tokens: 1 at perfect certainty, V at the uniform baseline.
- Bits view.** $\mathcal{L}_{\text{bits}} = \log_2 \text{PPL}$ is the compression rate; halving the loss does not halve perplexity, it takes its square root.

Perplexity is only comparable across models with the same tokenizer. The product in (2) runs over *tokens*, and T counts tokens, so a model that splits text into more (shorter) sub-words spreads the same sentence over more, individually-easier factors. A word-level and a byte-pair model can report wildly different perplexities on identical text purely because of how they segment it. Always normalise (e.g. to bits-per-byte) before comparing across tokenizers, and never read a raw PPL number without knowing the vocabulary it was computed on.

6 Where this sits in the track

Perplexity is the standard yardstick for the autoregressive loss of Topic 1, and the number you watch fall during GPT-2 pretraining and report on held-out data. Topic 3 turns from *measuring* the distribution p_t to *sampling* from it (temperature, top- k , top- p); the entropy of p_t introduced there is exactly the per-step quantity whose exponential, averaged, gives perplexity.

A Reproducible (NumPy)

Inputs: $p = (0.610405, 0.558696, 0.475332, 0.464426)$, $T = 4$. Intermediates: $\prod p = 0.075285$; $\mathcal{L} = 0.646619$ nats; $\mathcal{L}_{\text{bits}} = 0.932874$; PPL = 1.909076. Uniform check: $V = 5 \Rightarrow \text{PPL} = 5$.

```
import numpy as np
p = np.array([0.610405, 0.558696, 0.475332, 0.464426])
T = len(p)
L_nats = -np.mean(np.log(p))           # 0.646619
L_bits = -np.mean(np.log2(p))          # 0.932874 = L_nats/ln 2
PPL     = np.exp(L_nats)                # 1.909076
PPL2    = (np.prod(p))**(-1/T)         # 1.909076 (geom-mean form)
# uniform baseline: every p_t = 1/V -> PPL = V
V = 5
print(L_nats, L_bits, PPL, PPL2, np.exp(-np.log(1/V))) # last -> 5.0
```